Real-Time Embedded Systems
Final Assignment: Final Assignment

Pierre — AERO 4
IPSA

April 20, 2026

Abstract

This report addresses the final assignment of the Real-Time Embedded Systems course. The worst-case execution time (WCET) of a large-integer multiplication task τ_1 is estimated empirically from 5000 runs of a C++ routine. The resulting task set is then analyzed for schedulability under a non-preemptive, job-level policy. We derive an algorithm that builds a non-preemptive schedule minimizing the total waiting time over the hyperperiod, verify that every job meets its deadline.

1 WCET of the multiplication task

Task τ_1 multiplies five large, randomly generated integers. Because the multiplication kernel is written in C++, it is compiled once and then invoked $N = 5000$ times from Python through a dedicated module (`calcul_temps_multiplication`). For every call the wall-clock execution time is recorded, which produces an empirical distribution of the execution time c_1.

The summary statistics obtained on the target platform are analysed with `percentile` function and reported in Table 1.

Table 1: Execution-time statistics of τ_1 (5000 runs).

Statistic	Value (s)
Minimum	0.0086
First quartile Q_1	0.0125
Median Q_2	0.0142
Third quartile Q_3	0.0172
Maximum	0.1366

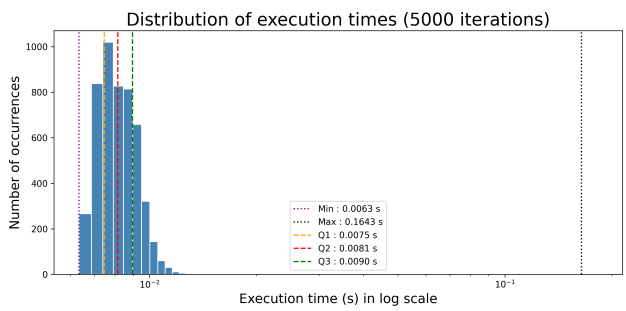

Figure 1: Distribution of the execution times

To have something with the same order of magnitude of the future execution times, I multiply the values by 10. Then, to have my WCET, I've took the Maximum Value (= 1.366 seconds) and to have an integer, I've selected the upper integer using `np.ceil` function. In this case,

$$\text{WCET} = 2$$

2 Task set and assumptions

The complete task set, with C_1 substituted, is given in Table 2. Tasks are periodic with implicit deadlines T_i.

Table 2: Task set with $C_1 = 2$.

Task	C_i	T_i	Jobs per hyperperiod
τ_1	2	10	8
τ_2	3	10	8
τ_3	2	20	4
τ_4	2	20	4
τ_5	2	40	2
τ_6	2	40	2
τ_7	3	80	1

There is no priority given. So we can use Round-Robin, Rate Time and Earliest Dead Line First algorithms.

3 Schedulability analysis

Processor utilization. As we suppose we are with a uniprocessor, the first condition (necessary one) for these 3 algorithms is:

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i} \leq 1.$$

With $C_1 = 2$,

$$U = \tfrac{2}{10} + \tfrac{3}{10} + \tfrac{2}{20} + \tfrac{2}{20} + \tfrac{2}{40} + \tfrac{2}{40} + \tfrac{3}{80} = 0.8375,$$

which leaves about 16% of idle bandwidth — the task set is loaded but not over-utilized.

3.1 Round-Robin

In a cyclic, non-preemptive Round Robin policy every task gets a turn in a fixed order $\tau_1 \to \tau_2 \to \cdots \to \tau_7$, regardless of deadlines. The function `round_robin` produces a feasible schedule over the hyperperiod (29 jobs placed without overlap), but RR ignores periods entirely: urgent jobs such as τ_1 and τ_2 (with $T = 10$) may wait behind one full round before being served, which is unacceptable for real-time guarantees in a more loaded task set. We therefore retain RR only as a baseline.
So with the Round-Robin algorithm, we have this schedule:

```
  J11 J21 J31 J41 J51 J61 J71 J12 J22 - J32
J42 J13 J23 - J14 J24 - J33 J43 J52 J62 J15
J25 J16 J26 - J34 J44 J17 J27 - J18 J28 -
```

3.2 Rate Monotonic

The classic Liu & Layland bound gives $U_{\mathrm{LL}}(7) = 7(2^{1/7} - 1) \approx 0.729$. Since $U = 0.8375 > U_{\mathrm{LL}}(7)$, the sufficient test *fails*. A direct simulation in the notebook (`rate_monotonic`) confirms the verdict: at least one low-priority job misses its deadline and the function returns:

```
''NOT schedulable''
```

3.3 Earliest Deadline first

At every scheduling instant the job with the smallest absolute deadline is dispatched (ties broken by task index) and runs to completion. A non-preemptive policy is viable because the longest task ($C_2 = 3$) is shorter than the shortest deadline ($D_1 = D_2 = 10$), so blocking never exceeds the relative deadline of the blocked task. The notebook function `earliest_deadline_first` places all 29 jobs of the hyperperiod within their deadlines, and every response time $R_{i,k} = f_{i,k} - r_{i,k}$ satisfies $R_{i,k} \le D_i$; the worst-case response times are $R_{\max} = (3, 6, 8, 10, 11, 18, 21)$ for $\tau_1, \ldots, \tau_7$, all below their deadlines $(10, 10, 20, 20, 40, 40, 80)$. We adopt this policy in the remainder of the report.
So we have this schedule:

```
    J11 J21 J31 J41 J51 J12 J22 J61 J71 - J13
J23 J32 J42 - J14 J24 - J15 J25 J33 J43 J52
J16 J26 J62 - J17 J27 J34 J44 - J18 J28 -
```

4 Algorithm & complexity

Algorithm. The scheduler advances time step by step over the hyperperiod $H = 80$. At each step t it looks at the jobs already released and still pending, picks the one with the closest deadline, and lets it run (uninterrupted) for its C_i time units. If no job is ready, the processor stays idle for one unit and the check is repeated at $t + 1$.

Result. Over one hyperperiod the processor is busy for $\sum_i (H/T_i)\, C_i = 67$ units out of 80, leaving 13 idle units ($= H(1 - U)$). The total waiting time, i.e. the sum over all jobs of the delay between release and start of execution, is

$$W_{\mathrm{tot}} = 140.$$

Because the processor idles only when *no* job is pending, idle time is pushed as late as possible and therefore maximized — which is consistent with minimizing W_{tot}.

Complexity. Let $N = \sum_i H/T_i$ be the total number of jobs per hyperperiod ($N = 29$ here). The scheduler performs at most N dispatch decisions, each of which picks the minimum-deadline job among the pending ones. With a priority queue the cost is $\mathcal{O}(N \log N)$; with a simple list scanned linearly it becomes $\mathcal{O}(N \cdot n)$, where n is the number of tasks.

5 Conclusion

Starting from an empirical WCET estimation of a C++ multiplication kernel and a 50% safety margin leading to $C_1 = 2$, we have shown that the seven-task set is schedulable on a uniprocessor ($U = 0.8375$), that Rate Monotonic is rejected by the Liu & Layland test and fails in simulation, and that non-preemptive EDF at the job level meets every deadline over the hyperperiod $H = 80$, with a total waiting time $W_{\mathrm{tot}} = 140$ and 13 idle units ($= H(1 - U)$). The algorithm runs in $\mathcal{O}(N \log N)$ per hyperperiod, where $N = 29$ is the total number of jobs.

The full code (multiplication kernel, statistics, schedule generators) is available on the accompanying Git repository:
https://github.com/guenopierre/In422_Final_Assignment.git.